



Face Multiclassifier Project Report

Thomas More University of Applied Sciences
AI-Frameworks

Teaching and Supervising Professor:
Daiane Saibert

Submitted by
Matthias Hefel

Duffel, January 2025

Contents

Contents	2
Introduction	3
Data	3
Kaggle Competition	3
Data Cleaning	3
Label-Data Cleaning	3
Image-Data Cleaning	3
Face Detection	4
Creating the train-set	4
Example detection process	5
Choosing the right model	7
Multiple face recognizers	7
Models	8
Haarcascade Frontal Face	8
MTCNN	8
RetinaFace	8
Face Recognition	9
Models	9
OpenCV LBPH Face Recognizer	9
Training	9
Results	10
Conclusion	10
OpenCV Fisher Face Recognizer	10
Training	10
Results	10
Conclusion	10
CNN with ResNet502V2 and Imagenet	10
Conclusion	11
Facenet	11
Training Entire Model	12
Fine Tuning Model	12
Results	13
Why Chosen	14
Summary of Approaches	14
ResNet502V2 and Imagenet	14
Sourcecode	14
Conclusion	14
Takeaways	14
Challenges	15
AMD GPU Support	15
Conflicting Package Versions	15
Sources	16

Introduction

This project's goal is to create and test machine learning models in order to recognize classmates in an image.

Goals were to clean data, preprocess images, detect faces and create a model that can recognize faces. And then put this into a pipeline, which takes an image and gives you which people are in the image, from left to right.

Data

The data consisted of 831 images of students from our course. We took them with our phones in one session, just outside of the university. We took all kinds of different photos, with different angles and different combinations of people in the photos.

We then manually labeled all images in a csv file, which was good, but not 100% accurate.

Kaggle Competition

Results were submitted on kaggle, to compete and compare results:

<https://www.kaggle.com/competitions/who-is-this-student>

Data Cleaning

Label-Data Cleaning

The original label data had inconsistent formatting, like upper-case and lower-case, as well as inconsistent spaces. In addition to inconsistent formatting, the label data had many misspelled names.

In the first step, the labels were converted into a consistent format, by converting everything into lowercase and removing spaces.

Then I checked for all misspelled names and manually created a correction table. This table was applied to the labels and replaced all misspelled names with the corrected ones to ensure consistent labeling and only having 14 classes.

This corrected label file is then saved as a .csv file.

Image-Data Cleaning

Images were in different formats like .jpg .png .heic or mp4 and various different resolutions.

To create a uniform dataset for training, all images were transformed into a consistent file format. I chose .jpg for this, because it is easy to handle and great for real life images.

Although .jpg files are compressed, I do not think it is necessary to keep the additional detail from .png images.

Handling video files in .mp4 images was tricky, since not all people are visible in all frames. Despite this constraint I chose to only extract the first frame and convert it to .jpg for simplicity's sake and since there are not many video files in the data-set.

As the final step, all images were saved in a new folder as 640x480 pixel jpg files to ensure consistent formatting and ensure that the next preprocessing steps like face detection and cropping were consistent and effective.

Face Detection

I have tested several face detectors to detect and crop faces, like Haarcascade, MTCNN and finally RetinaFace.

My first attempt used Haarcascade Frontal Face as the face detector, but it showed rather poor results, since many images had not only frontal, but also side or top view images of the faces, which this detector was not able to pick up. Then I switched to MTCNN, which was marginally better at detecting faces in different lighting conditions and viewports. But after many attempts and reading a few articles, RetinaFace proved to be the most accurate face detector, for this project. Although Retinaface was really good at detecting faces, it was not able to detect a face in every image, and since every image contains at least one face, I created a fallback mechanism that crops the center of the image, if RetinaFace was not able to detect any faces.

Creating the train-set

For creating the testset I used the face detector to cut out all the faces in the image and then used the image-label-map to sort the cropped faces into their respective folders. This was relevant to create usable test data and determine if some people were over or underrepresented in the dataset. Since the dataset was relatively small (around 800 images), it was possible to manually check if the sorting worked as expected. Through some testing I found out that it was crucial to have a somewhat low threshold e.g. it is important to detect as many faces as possible (without false detections) to make the process efficient. The training data creation was really poor when not the correct number of faces in an image were detected, because the cutout faces were then “misabeled” and put into the wrong folder for training.

Example detection process

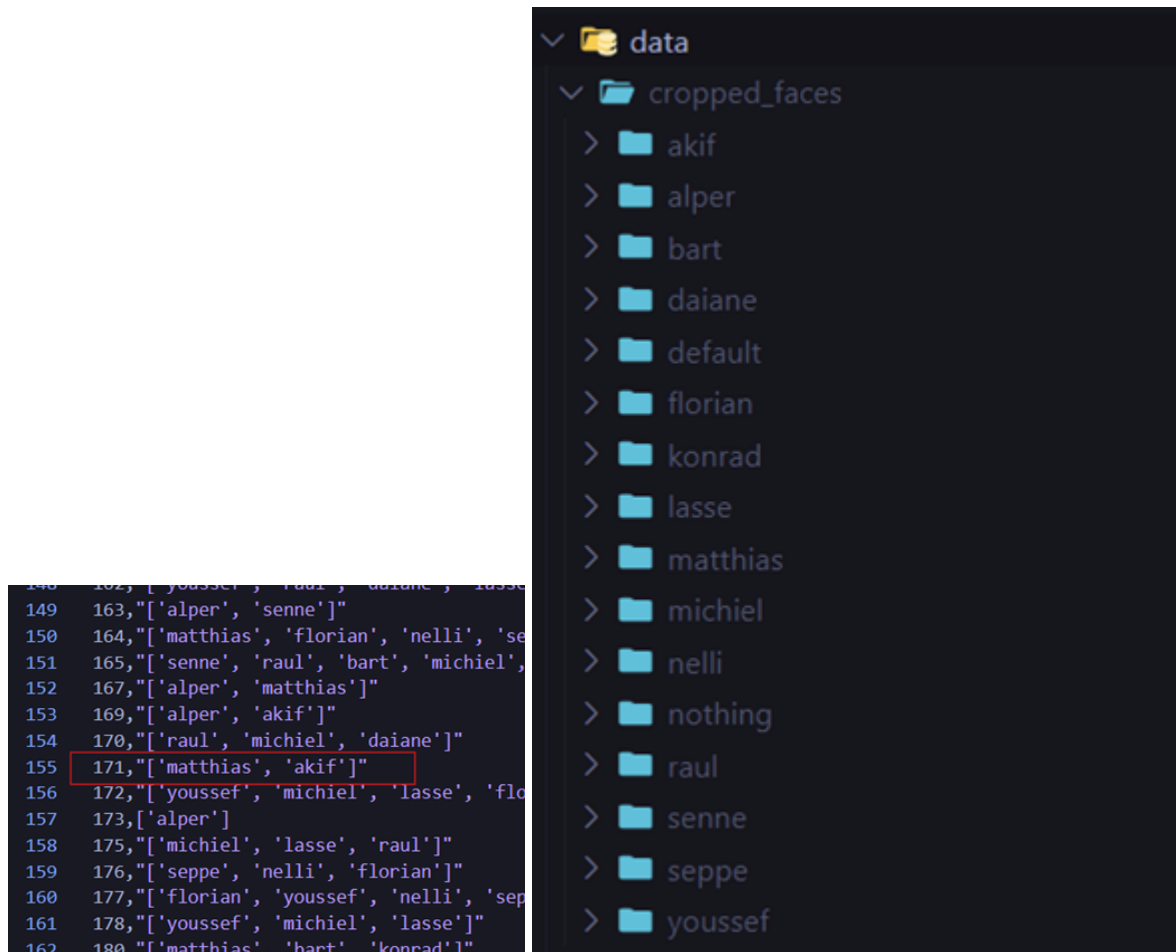
1. Cleaned Image 0171.jpg (640x480) as Input.



2. RetinaFace.detect_faces(0171.jpg) -> gives 2 cutouts for the faces with coordinates of the bounding boxes around each face:



3. These images then get sorted by their X-Coordinate of the bounding box (e.g. sorting the faces from left to right, to align with the labeling file).
4. Then the corresponding labels of the label file get checked from left to right and the images get saved into their corresponding folder.



So in the end there is a folder with subfolders for each class/person containing cropped faces of each person in (224x224) resolution jpg files.

Choosing the right model

Choosing the right model was rather straightforward but took some time to get right. At first I chose the Haarcascade Frontal face from OpenCV as a face detector, but came to the conclusion that it is missing a lot of faces. This is why I then started my research and came across MTCNN and RetinaFace, which both are way better face detectors. After doing some testing and reading It became certain that RetinaFace would outperform MTCNN, which is why I chose it as the face detector for cropping the faces for the test as well as training set.

Metric	Haarcascade Frontal Face	MTCNN	RetinaFace
Accuracy	Low	Medium	High
Robustness to Variations	Low (only frontal)	Medium	High (works for diverse angles and lighting)
Detection speed	Fast	Medium	Medium
Landmark Detection	No	Yes	Yes
Threshold Tuning Options	Limited	Moderate	Moderate
Fallback Mechanism Integration	Challenging	Easy	Easy
Performance on Dataset	Many missed faces	Better but does not detect all faces	Consistently detected most faces

Multiple face recognizers

I was thinking about utilizing multiple face recognizers to detect more faces. So that for example, if RetinaFace would not be able to detect a face, MTCNN would take over and scan as well, but instead of this approach, I lowered the threshold of RetinaFaces `detect_faces()` function and added a default crop, that would crop the middle of the image with a slight zoom, if nothing was detected. This still increased the submissions, without adding much complexity.

Models

Haarcascade Frontal Face

“Object Detection using Haar feature-based cascade classifiers is an effective method proposed by Paul Viola and Michael Jones in the 2001 paper, “Rapid Object Detection using a Boosted Cascade of Simple Features”. It is a machine learning based approach in which a cascade function is trained from a lot of positive and negative images. It is then used to detect objects in other images.”

(https://docs.opencv.org/4.x/d2/d99/tutorial_js_face_detection.html)

Implemented in OpenCV, it is very easy to integrate and get running. It is Fast, lightweight and mostly focuses on detection of frontal faces.

It has poor detection accuracy for other views, than frontal, such as side, or top views. This is why it was not suitable for use in this project, since the dataset had many angles and different conditions.

MTCNN

“MTCNN is a robust face detection and alignment library implemented for Python >= 3.10 and TensorFlow >= 2.12, designed to detect faces and their landmarks using a multitask cascaded convolutional network” (<https://github.com/ipazc/mtcnn>)

MTCNN is a deep learning-based face detection model, that detects faces and landmarks.

Slower than Haarcascade, but more accurate in detecting faces from different angles.

RetinaFace

“RetinaFace is a deep learning based cutting-edge facial detector for Python coming with facial landmarks. Its detection performance is amazing even in the crowd as shown in the following illustration. RetinaFace is the face detection module of insightface project. The original implementation is mainly based on mxnet.” (<https://github.com/serengil/retinaface>)

State-of-the-art single-stage dense face localization model. Excels in detecting faces from different angles, poses, and bad lighting conditions.

Although the best, not perfect.

After finding out that RetinaFace appeared to detect the most faces accurately, I started experimenting with the `.detect_faces()` threshold, which determines how strict/loose the detection is. I Have found out that going to a lower threshold from the default 0.9 to between 0.6 and 0.4 gave the best results and improved my kaggle submissions. I then settled for a threshold of 0.5 which gave good results.

Threshold	Total faces detected	0 faces detected (fallback crop)	Average faces per image with detection
0.9 (default)	152	12	1.73
0.7	163	11	1.83
0.6	168	11	1.85
0.5	167	11	1.88
0.4	171	10	1.90
0.35	175	10	1.94
0.1	195	6	2.07

Detecting the right amount of faces proved to be very important for the test-set, but not so impactful for creating the training-set. When creating the training data, it is more important the cropped image is actually a face, rather than cutting out the perfect amount of faces. That is why I implemented the threshold of 0.5, as well as the fallback crop in the middle of the image for creating the test-set and the default 0.9 threshold for creating the training data.

Face Recognition

A crucial part in this project was the creation of a robust and accurate face recognition model. The task of this model is to recognize which of the 14 people the cropped face belongs to. To achieve this, I experimented with a bunch of different models and approaches, including traditional approaches like LPPH and Fisher Face Recognizer, as well as deep learning-based models like custom CNN and Facenet.

Models

OpenCV LBPH Face Recognizer

“For the face recognition process, Local Binary Pattern Histogram(LBPH) algorithm is applied. The LBP operator uses Local Binary Patterns to decrease the local spatial distribution of a face image. The LBP operator is a collection of binary pixel value ratios in the center at regular pixel intervals and is around eight pixels”

(https://sist.sathyabama.ac.in/sist_naac/documents/1.3.4/1822-b.tech-it-batchno-367.pdf)

Training

The training process was straightforward. I used the cropped faces as input and trained the LBPH model using OpenCV. Each face was assigned a label corresponding to one of the 14 classes/people.

Results

The results were rather lackluster. The model was simple to set up/implement and fast to train, it could recognize very clear images of the faces, it lacked when trying to recognize dark or small images. This resulted in poor accuracy in my tests.

Conclusion

It was the first implementation that I tried and I managed to get it working really fast. Training also was fast and it was able to recognize faces, but pretty poorly. The model lacked the robustness that was needed for this project, which is why I chose not to use it in the end.

OpenCV Fisher Face Recognizer

"Fisherface is one of the popular algorithms used in face recognition, and is widely believed to be superior to other techniques, such as eigenface because of the effort to maximize the separation between classes in the training process."

"Image recognition using fisherface method is based on the reduction of face space dimension using Principal Component Analysis (PCA) method, then apply Fisher's Linear Discriminant (FDL) method or also known as Linear Discriminant Analysis (LDA) method to obtain feature of image characteristic."

(<https://iopscience.iop.org/article/10.1088/1742-6596/1028/1/012119/pdf>)

Training

Similar to LBPH, the training was straightforward and I used the cropped faces to train the model. The implementation was very easy with OpenCV and the transition from LBPH to Fisher Face Recognizer was not too difficult to implement.

Results

Although I read that Fisher Face Recognizer should perform a bit better than LPBH in face recognition tasks, in my tests, it performed worse than LBPH and was therefore eliminated rather quickly.

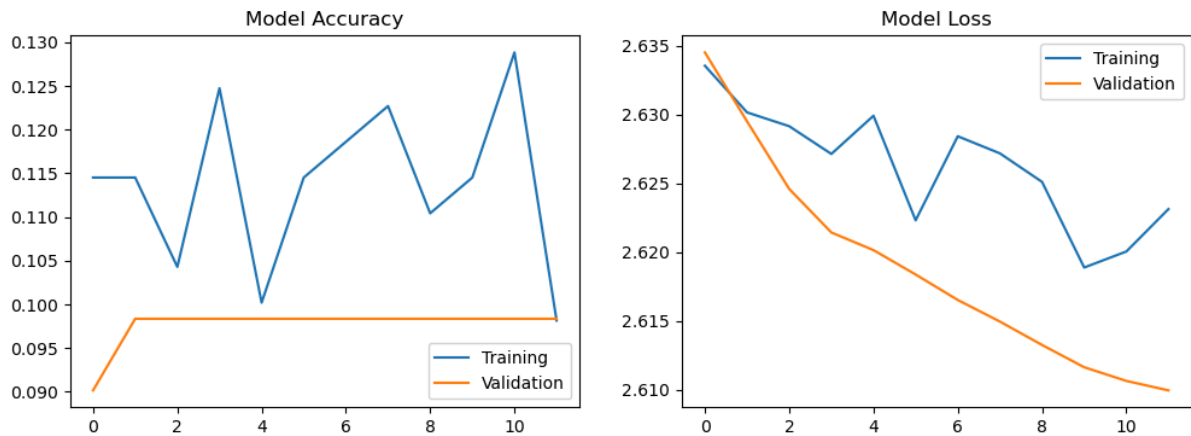
Conclusion

I could have experimented with parameters and more sophisticated approaches, but I did not deem it worth investing the time to improve this model.

CNN with ResNet502V2 and Imagenet

I used ResNet502V2 with Imagenet weights, with a partially trainable base model and added a series of (Batchnormalization -> Dense -> Activation -> Dropout) layers and a last layer for classification of the 14 classes, to the base model.

Although implementing a bunch of callback functions, like early stopping, reduce learning rate on plateau and model checkpoint, I could not get the model to converge and the accuracy was rather horizontal around 11%.



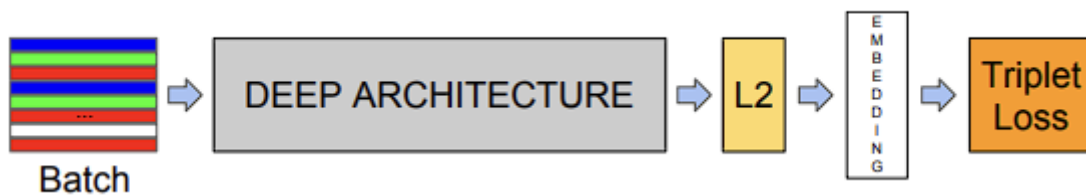
Conclusion

Although I probably made a few mistakes along the way of this approach, which is why it did not work, I learned a bunch of techniques, like data augmentation, callback functions and how to add layers to the model, which came in handy, in the approach with Facenet.

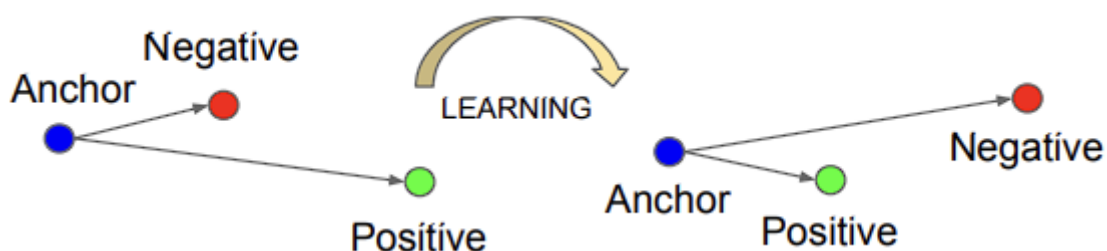
Facenet

“FaceNet uses a deep convolutional network” “Our method is based on learning a Euclidean embedding per image using a deep convolutional network. The network is trained such that the squared L2 distances in the embedding space directly correspond to face similarity: faces of the same person have small distances and faces of distinct people have large distances.”

“Model structure. Our network consists of a batch input layer and a deep CNN followed by L2 normalization, which results in the face embedding. This is followed by the triplet loss during training.”



“The Triplet Loss minimizes the distance between an anchor and a positive, both of which have the same identity, and maximizes the distance between the anchor and a negative of a different identity.”



(https://www.cv-foundation.org/openaccess/content_cvpr_2015/papers/Schroff_FaceNet_A_Unified_2015_CVPR_paper.pdf)

Training Entire Model

I Initially tried to train the entire Facenet model on the dataset, with just the added layers for the classification to 14 classes. This approach worked, but was prone to overfitting, due to it being a relatively small dataset. In my case, it tended towards classifying faces as “Akif”. The enormous training time made it hard for me to iterate. This then led me to the approach of just fine tuning the model and leaving the base layers as they are.

Fine Tuning Model

I fine tuned the pre-trained Facenet model by adding custom classification layers on top of the existing architecture.

The base layers were frozen and only the custom layers were trained to adapt to the dataset.

layer	size-in	size-out	kernel	param	FLPS
conv1	220×220×3	110×110×64	7×7×3, 2	9K	115M
pool1	110×110×64	55×55×64	3×3×64, 2	0	
rnorm1	55×55×64	55×55×64		0	
conv2a	55×55×64	55×55×64	1×1×64, 1	4K	13M
conv2	55×55×64	55×55×192	3×3×64, 1	111K	335M
rnorm2	55×55×192	55×55×192		0	
pool2	55×55×192	28×28×192	3×3×192, 2	0	
conv3a	28×28×192	28×28×192	1×1×192, 1	37K	29M
conv3	28×28×192	28×28×384	3×3×192, 1	664K	521M
pool3	28×28×384	14×14×384	3×3×384, 2	0	
conv4a	14×14×384	14×14×384	1×1×384, 1	148K	29M
conv4	14×14×384	14×14×256	3×3×384, 1	885K	173M
conv5a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv5	14×14×256	14×14×256	3×3×256, 1	590K	116M
conv6a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv6	14×14×256	14×14×256	3×3×256, 1	590K	116M
pool4	14×14×256	7×7×256	3×3×256, 2	0	
concat	7×7×256	7×7×256		0	
fc1	7×7×256	1×32×128	maxout p=2	103M	103M
fc2	1×32×128	1×32×128	maxout p=2	34M	34M
fc7128	1×32×128	1×1×128		524K	0.5M
L2	1×1×128	1×1×128		0	
total				140M	1.6B

(Existing architecture, Base layers)

```
x = base_model.output
x = Flatten()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output_layer = Dense(num_classes, activation='softmax')(x)
```

(Added layers for classification)

I added an ImageDataGenerator to augment the data in various ways, such as rotation, width shift, height shift, zoom, brightness adjustment and horizontal flips. I also made sure to normalize all the pixel values by dividing them by 255 to help the model train.

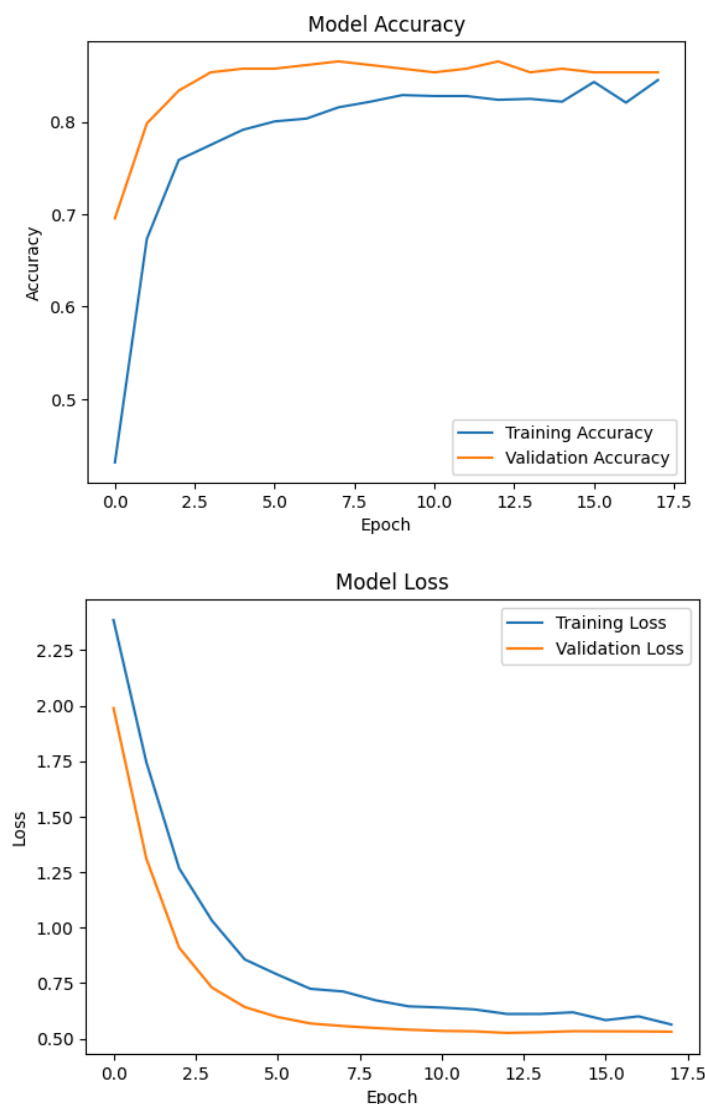
For the training process, I added an early stopping callback with a patience of 5 epochs and a callback function to reduce the learning rate on a plateau.

The training-time was manageable and i could iterate over a few parameters, without having to wait so long for the model to train as with the fully trained model.

Results

The fine-tuned model was clearly faster to train and had better accuracy, which is why I chose this model in the end.

With the chosen settings, the model converged after 18 epochs, with an accuracy of 82.41% and a validation accuracy of 85.38%.



Why Chosen

Facenet is a state of the art face recognition model and has a great robustness and the best accuracy out of all of them. The pre-trained embeddings of the Facenet model captured rich facial features, which worked great with different cropped images like in low lighting and side-views of the face. Fine-tuning the model improved the accuracy by a lot without requiring too much training time, which was crucial, since I only had access to my CPU. Training the entire model took around 3-5 times longer, than only fine-tuning.

Summary of Approaches

Model	Accuracy	Robustness	Training Time	Computational Cost	Final Decision
LBPH Face Recognizer	Medium	Low	Fast	Low	Not used
Fisher Face Recognizer	Low	Low	Medium	Low	Not used
ResNet502V2 and Imagenet	Very Low	Very low	Very High	High	Not used
Facenet (Full training)	High	High	Very High	Very High	Passable
Facenet (Fine Tuning)	Very High	Very High	High	medium	Chosen

Sourcecode

All source code for this project can be found on my github page:

https://github.com/NeYoNmoos/Face_Multiclassification_Thomas-More

Conclusion

Takeaways

It is possible with a short “data gathering session”, like we did in the beginning of the semester, to create enough training data to train a machine learning model that can fairly accurately predict, which people are in which picture.

Once a pipeline, from raw data to training and evaluating the model is set up, one of the main aspects is experimenting with different models, different parameters and ultimately finding the right combination to achieve the best result.

It is fun to work on a kaggle competition and see how other peoples' models perform in relation to their own. This helped me know what is possible and motivated me to tune my approach for a better and better result. I will definitely consider taking part in other kaggle competitions, if i find some free time for it.

Challenges

AMD GPU Support

Since there is no Cuda support for AMD graphics cards, and the other solutions for AMD graphics cards did not work with my setup, I had to rely on the CPU only.

This restriction drastically increased computation times, for processing images, detecting faces, training models in addition to other CUDA errors with tensorflow.

Conflicting Package Versions

Training and using different models in the same virtual environment proved to be a challenge when it came to handling packages that work for all implemented models. Packages like tensorflow, tf-keras and retina-face gave me a bunch of conflicting errors, which is why I adapted to using different virtual environments for face detection and for face recognition.

Sources

RetinaFace:

(<https://github.com/serengil/retinaface>)

RetinaFace: Single-stage Dense Face Localisation in the Wild:

(<https://arxiv.org/pdf/1905.00641>)

Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks:

(<https://arxiv.org/pdf/1604.02878>)

Comparative Analysis of State-of-the-Art Face Recognition Models: FaceNet, ArcFace, and OpenFace Using Image Classification Metrics:

(https://www.researchgate.net/publication/377935125_Comparative_Analysis_of_State-of-the-Art_Face_Recognition_Models_FaceNet_ArcFace_and_OpenFace_Using_Image_Classification_Metrics)

Face Detection using Haar Cascades:

(https://docs.opencv.org/4.x/d2/d99/tutorial_js_face_detection.html)

What is the best face detector:

(<https://medium.com/pythons-gurus/what-is-the-best-face-detector-ab650d8c1225>)

Facenet, Openface, Arcface comparison:

(https://www.researchgate.net/publication/377935125_Comparative_Analysis_of_State-of-the-Art_Face_Recognition_Models_FaceNet_ArcFace_and_OpenFace_Using_Image_Classification_Metrics#:~:text=Even%20while%20ArcFace%20and%20OpenFace,applications%20where%20accuracy%20is%20crucial)

Facenet, LBPH, face_recognition comparison:

(<https://www.hackerox.com/blog-detail/attendance-tracker-face-off-which-face-recognition-model-reigns-supreme>)

Facenet: A Unified Embedding for Face Recognition and Clustering

(https://www.cv-foundation.org/openaccess/content_cvpr_2015/papers/Schroff_FaceNet_A_Unified_2015_CVPR_paper.pdf)

Face Recognition using Fisherface method:

(<https://iopscience.iop.org/article/10.1088/1742-6596/1028/1/012119/pdf>)